INSTITUTO SUPERIOR TECNOLÓGICO QUITO

Tercer Módulo – Python, Tipos de Datos - Data Types

Carlos Alberto Pacho

Comentarios

¿Qué son?

Texto contenido en ficheros de Python que es ignorado por el intérprete; es decir no es ejecutado.

¿Cómo y cuándo se utilizan?

- Se puede utilizar para documentar código y hacerlo más legible.
- Preferiblemente trataremos de hacer fácil de entender y que necesite pocos comentarios, en lugar de vernos forzados a recurrir a los comentarios para explicar el código.

Tipos de comentarios

Comentarios de una línea

Texto precedido por '#' Se suele usar para documentar expresiones sencillas

[1]:
```
# Esto es una instruccion print
print("Hola word") # Esto es una instruccion print
```

```
Hola word
```

Comentarios de varias líneas

- Texto encapsulado en triple comillas (que pueden ser simples como dobles)
- Se suele usar para documentar bloques de código más significativo

[2]:
```
# crea una funcion de multiplicación
def producto(x, y):
    '''
    Esta función recibe dos números como parámetros y devuelve como resultado el producto de los mismos

    Acceso: una variable donde guarda el producto de los numeros recibidos
    x = x * y
    return x
    '''
# llamo a la función
producto(3, 7)
```

[3]: 21

Literales, variables y tipos de datos básicos

De forma muy genérica el ejecutarse un programa en Python se realizan operaciones sobre objetos

Estos dos términos son conceptualmente

Objetos: Cualquier tipo de datos (números, caracteres o datos más complejos).

- Operaciones: Como manipulamos estos datos. +,-,*,/

Ejemplo:

[4]:
```
4 * 3
```

[4]: 12

Literales

- Python tienen una serie de tipos de datos integrados en el propio lenguaje.
- Los literales son expresiones que generan objetos de estos tipos.
- Estos objetos, según su tipo pueden ser
  - Simples o compuestos
  - Mutables o inmutables

Literales simples

- Enteros
- Decimales o punto flotante
- Booleanos

[5]:
```
print(4)                    #numero entero
print(4.3)                  #numero punto flotante
print("Hola word")          #objeto de tipo string
print(False)                #Objeto de tipo booleano
```

```
4
4.3
Hola word
False
```

Literales compuestos

- Tuplas
- Listas
- Diccionarios
- Conjuntos

[6]:
```
print([1, 2, 3, 5]) # lista - mutable
print({"nombre": "John Doe", "edad": 30}) # Diccionario - mutable
print((1, 2, 3, 5)) # tuplasta - mutable
print({1, 5}) # tupla - inmutable
[1, 2, 3, 5]
{'nombre': 'John Doe', 'edad': 30}
(1, 2, 3, 5)
{1, 5}
```

[6]: {1, 4}

Variables

- Referencias a objetos.
- Las variables y los objetos se almacenan en diferentes zonas de memoria.
- Las variables siempre referencian a objetos y nunca a otras variables.
- Objetos si que pueden referenciar a otros objetos. Ejemplo: listas.
- Sentencia de asignación:

```
<v>
```

[7]:
```
# Asignación de variables
a = 5
print(a)
```

```
5
```

[8]:
```
a = 5 # entero
b = 4.4 # coma flotante
c = True # Boolean
d = 10 + 5j # numero complejo
e = True #False # boolean
f = None # none
# visualizar valor de las variables y su tipo
print(a)
print(type(a))
print(b)
print(type(b))
print(c)
print(type(c))
print(d)
print(type(d))
print(e)
print(type(e))
print(f)
print(type(f))
```

```
5
<class 'int'>
4.4
<class 'float'>
True
<class 'str'>
(10+5j)
<class 'complex'>
True
<class 'bool'>
None
<class 'NoneType'>
```

- Las variables no tienen tipo.
- Las variables apuntan a objetos que si lo tienen.
- Dado que Python es un lenguaje de tipado dinámico: la misma variable puede apuntar, en momentos diferentes de la ejecución del programa, a objetos de diferente tipo.

[9]:
```
a = 3
print(a)
print(type(a))
a = "Pablo García"
print(a)
print(type(a))
a = 6.5
print(a)
print(type(a))
```

```
3
<class 'int'>
Pablo García
<class 'str'>
6.5
<class 'float'>
```

- Garbage collection: Cuando un objeto deja de estar referenciado, se elimina automáticamente

Identificadores

- Podemos obtener un identificador único para los objetos referenciados por variables.
- Este identificador se obtiene a partir de la dirección de memoria.

[10]:
```
a = 3
print(a)
a = "Pablo García"
print(a)
a = 6.5
print(id(a))
```

```
3
140710417656800
140704661120
1893576243664
```

- Referencias compartidas: un mismo objeto puede ser referenciado por más de una variable.
  - Variables que referencian al mismo objeto tienen mismo identificador.

[11]:
```
a = 4567
print(id(a))
```

```
1883594665648
```

[12]:
```
b = a
print(id(b))
```

```
1883594665648
```

[13]:
```
a = 4567
print(id(a))
```

```
1883594667408
```

[14]:
```
a = 25
b = 25

print(id(a))
print(id(b))
print(id(25))
```

```
140710417656784
140710417656784
140710417656784
```

[15]:
```
# Si son los enteros "grandes" [-5, 256]
a = 259
b = 259

print(id(a))
print(id(b))
print(id(259))
```

```
1883594666176
1883594666144
1883594666144
```

- Referencia al mismo objeto a través de asignar una variable a otra.

[16]:
```
a = 400
b = a
print(id(a))
print(id(b))
```

```
1883594666464
1883594666464
```

- Las variables pueden aparecer en expresiones.

[17]:
```
a = 3
b = 5
print(id(a))
print (a + b)
```

```
140710417656800
8
```

Respecto a los nombres de las variables …

- No se puede poner números delante del nombre de las variables.
- Por convención, evitar CamelCase. Mejor usar snake_case usando '_' para separar palabras.
- El lenguaje diferencia entre mayúsculas y minúsculas.
- Deben ser descriptivos.
- Hay palabras o métodos reservados – Built ins y KeyWords
  - Ojo con reasignar un nombre reservado!

[18]:
```
potencia = 2
print(pow(3, 2))
```

```
9
```

[19]:
```
print(pow(3,2))

pow = 5 # built-in reasignado
print(pow)

print(pow(3,2))
```

```
9
5
```

No Code

```
TypeError:                                Traceback (most recent call last)
<ipynb_cell_id_1_>
<ipynb_cell_id_1_>
    2 pow = 5 # built-in reasignado
    3 print(pow)
----> 4 print(pow(3,2))

TypeError: 'int' object is not callable
```

[1]: `del pow(a, b)`

```
return a + b
print(pow(3,2))
```

Asignación múltiple de variables

[1]:
```
x, y, z = 1, 2, 3
print(x, y, z)
x = a, y, z, 7, "Python"
print(x)
print(type(x))
```

- Esta técnica tiene un uso interesante: el intercambio de valores entre dos variables.

[2]:
```
a = 1
b = 2

a, b = b, a
print(a, b)
```

[3]:
```
a = 1
b = 2

c = a
a = b
b = c
print(a, b, c)
```

Tipos de datos básicos

Bool

- 2 posibles valores: 'True' o 'False'

[1]:
```
a = False
b = True

print(a)
print(type(a))

print(b)
print(type(b))
```

- 'True' y 'False' también son objetos que se guardan en caché, al igual que los enteros pequeños.

[2]:
```
a = True
b = False

print(id(a))
print(id(b))

print(a is b)
print(a == b)
```

Números

[1]:
```
print(2) # Entero, sin parte fraccional.
print(3.4) # Numero en coma flotante, con parte fraccional.
print(5+4j) # Numero complejo.
print(1/2) # Numero racionales.
```

- Diferentes representaciones base 10, 2, 8, 16.

[2]:
```
a = 58 # decimal
a = 0b00111010 # binaria
a = 0o72 # octal.
a = 0x3A # hexadecimal

print(a == y == z == w)
```

Strings

- Cadenas de caracteres.
- Son secuencias: la posición de los caracteres es importante.
- Son inmutables: las operaciones sobre strings no cambian el string original.

[1]:
```
# "John Doe" Doe
print(a[0]) # Primer caracter del string.
print(a[-1]) # Último caracter del string.
print(a[0:3]) # Subcadena que va desde el segundo caracter (inclusive) hasta el octavo (exclusivo). Esta técnica se la co
print(a + "x") # Concatenación.
```

```
J
e
John "Jo" Doe
John "Jo" Doexx
```

Click to add a cell.